

Relatório 2 - Prática: Linguagem de Programação Python (I)

Igor Carvalho Marchi

1. Introdução

O objetivo deste relatório é descrever os conceitos fundamentais da linguagem Python explorados na atividade prática, abrangendo desde a organização de código em pacotes e módulos até os pilares da Programação Orientada a Objetos.

2. Desenvolvimento

O desenvolvimento da atividade permitiu compreender a versatilidade do Python através da experimentação de diversas estruturas de controlo e organização:

2.1 Organização e Modularização

- **Pacotes e Módulos:** Compreendi que pacotes funcionam como diretórios que agrupam módulos (arquivos .py) relacionados. Esta estrutura é vital para manter o código organizado e gerível à medida que o projeto cresce.
- **Importações:** O uso do comando import possibilita reutilizar funcionalidades prontas, como no caso da biblioteca functools, utilizada para acessar a função reduce, evitando reescrita de código e tornando o programa mais eficiente.

2.2 Estruturas de Dados e Lógica

Tipos básicos de variáveis

- **string:** Cadeias de texto.
- **integer:** Números inteiros.
- **float:** Números decimais.
- **booleana:** Valores booleanos (True ou False).

Conjuntos

```
conj = ({1, 2, 3, 3, 3, 3})
```

Os conjuntos são um tipo de estrutura que não permite valores duplicados, isto é mesmo que tenha 4 valores igual a 3, no painel será mostrado apenas {1, 2, 3}, ou seja, são utilizados para armazenar elementos únicos.

Lista

```
nums = [1, 2, 3]
```

Utilizam colchetes [], suportam valores duplicados e podem ser modificados.

```
nums.append(3) # adicione elementos na lista
```

append() serve para adicionar elementos na lista.

```
len(nums)
```

len() Serve para ler o tamanho da lista.

```
nums.insert(0, -200)
```

Insert() Serve para inserir um número em uma posição da lista.

Tuplas

```
nomes = ('Ana', 'Bia', 'Gui', 'leonardo', 'Ana')
```

Não é possível mudar as tuplas, elas devem ser preenchidas dentro de parênteses (), além disso diferente dos conjuntos elas aceitam valores repetidos. Após as aspas ' ' é necessário a presença de uma vírgula.

Dicionário

Dicionário	
<nome_variavel> : <valor atribuido>	
'Nome' : 'Igor'	
'Idade' : 19	
'AnoNascimento': 2006	

fonte: Criado pelo próprio autor

São estruturas de mapeamento com pares chave {}, com valores, acessados por meio de suas chaves, onde são declarados ex:

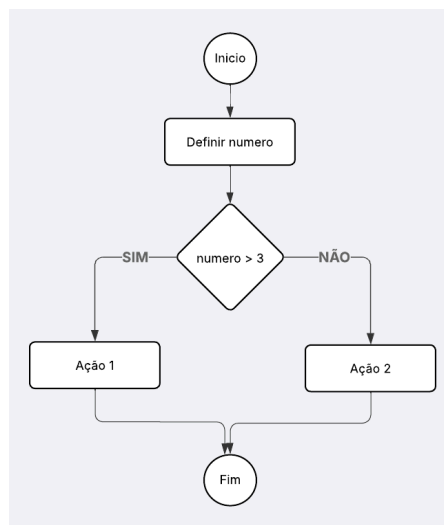
'nome' : 'Igor Carvalho Marchi'

Para buscar o nome Pedro Henrique basta chamar por 'nome'

2.3 Lógica de Controle e Repetição

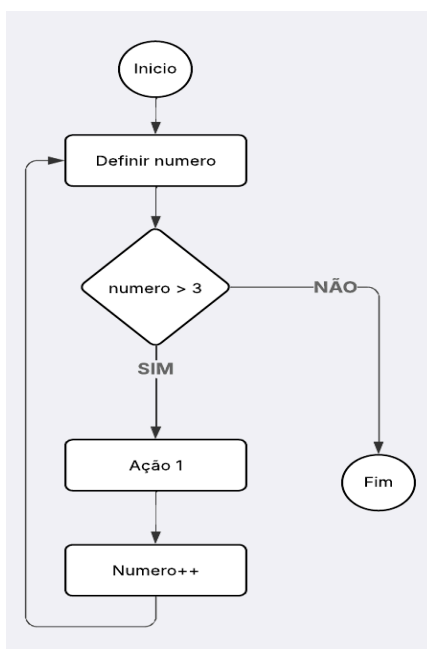
O fluxo de execução do programa é gerido por estruturas que permitem a tomada de decisões e a automação de tarefas repetitivas:

- **Condicional If:** Avalia uma expressão lógica. Se verdadeira, executa o bloco; caso contrário, pode seguir caminhos alternativos via elif ou else. É a base para criar inteligência e desvios no código.



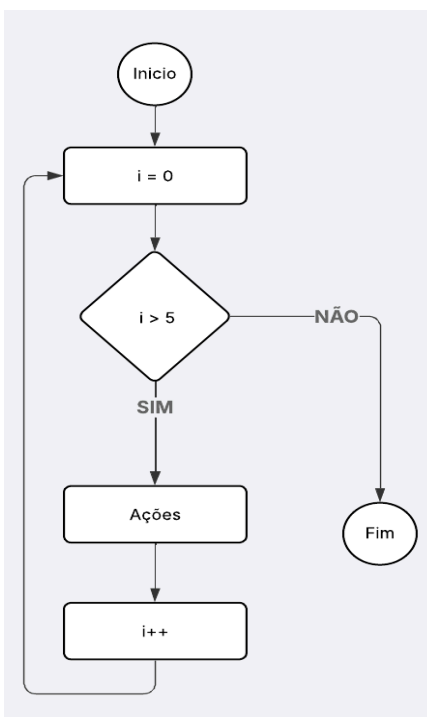
fonte: Criado pelo próprio autor

- **Laço While:** Repete um bloco de código enquanto uma condição for mantida. Como o Python não possui do while nativo, utilizamos while True com um break interno para garantir que o código seja executado pelo menos uma vez antes da validação.



fonte: Criado pelo próprio autor

- **Laço For:** Percorre sequências (listas ou intervalos range) de forma finita, sendo a estrutura mais estável para processar conjuntos de dados conhecidos.

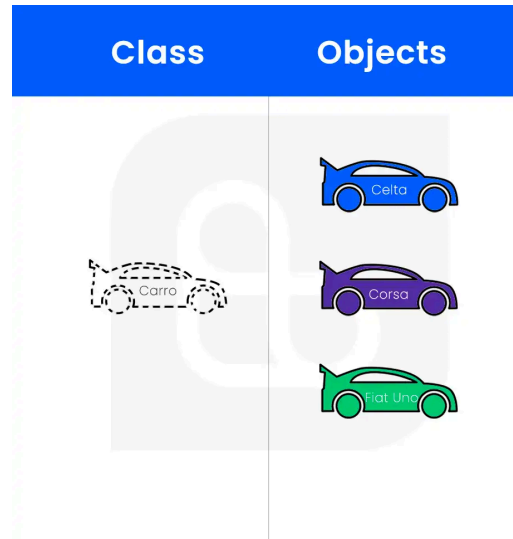


fonte: Criado pelo próprio autor

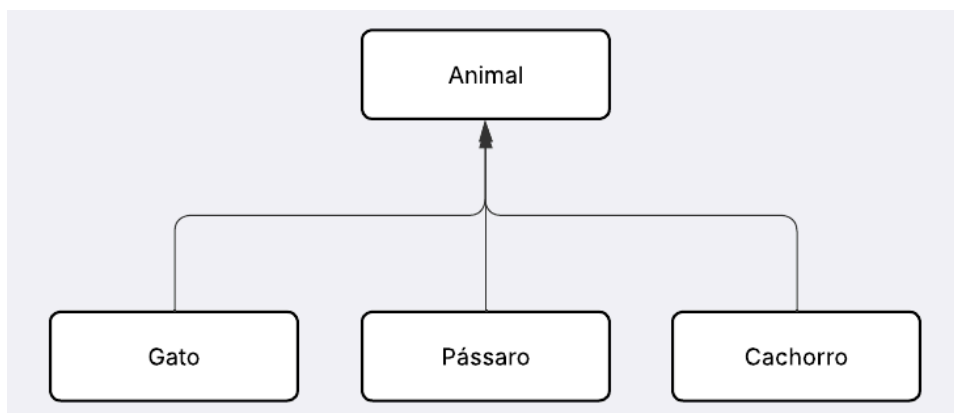
2.4 Programação Orientada a Objetos (POO)

A prática demonstrou como o Python implementa conceitos avançados de POO para simular entidades do mundo real:

- **Classes e Objetos:** Funcionam como moldes para a criação de instâncias com atributos e comportamentos específicos.



- **Encapsulamento e Decoradores:** O uso de `@property` e `@staticmethod` permite controlar o acesso aos dados e definir métodos que pertencem à classe, mas não dependem de uma instância específica.
- **Herança:** Observei como uma classe (ex: Ferrari) pode herdar e estender métodos de uma classe base (ex: Carro), facilitando a especialização do comportamento sem reescrever código base.



fonte: Criado pelo próprio autor

3. Conclusão

A realização desta atividade prática possibilitou a consolidação dos principais conceitos da linguagem Python, desde estruturas básicas até recursos avançados de Programação Orientada a Objetos. O teste prático demonstrou como Python é uma linguagem objetiva, organizada e poderosa, permitindo a criação de códigos reutilizáveis e bem estruturados.

Referencias

<https://www.youtube.com/watch?v=oUrBHiT-lzo>

<https://www.youtube.com/watch?v=iq7JLIH-sV0>

<https://didatica.tech/o-que-sao-modulos-e-pacotes-em-python-e-como-usar/#:~:text=Um%20m%C3%B3dulo%20%C3%A9%20basicamente%20um.ser%20facilmente%20gerenciados%20e%20reutilizados.>

<https://www.hashtagtreinamentos.com/estruturas-de-dados-em-python?>